1. Backlog da Sprint 2

Eu montaria o board do GitHub assim.

EPIC 1 — Ambiente

Tarefa 1 — Definir arquitetura do ambiente

Responsável: Pessoa 1

- definir máquina/VM do servidor;
- definir máquina/VM do agente;
- definir sistema operacional;
- definir rede entre elas;
- documentar IPs/portas;
- definir versão do Zabbix.

Definition of Done:

- arquitetura documentada;
- duas máquinas conseguem se comunicar;
- configuração registrada no README.

Tarefa 2 — Criar ambiente reproduzível

Responsável: Pessoa 2

Idealmente:

```
docker-compose.yml
README.md
.env.example
```

Se vocês conseguirem rodar Zabbix via Docker, melhor ainda para o trabalho acadêmico, porque facilita muito demonstrar que outra pessoa consegue reproduzir o ambiente.

Definition of Done:

> Uma pessoa que clonar o repositório consegue seguir o README e subir o ambiente.

Tarefa 3 — Configurar Zabbix Server

- Zabbix Server;
- frontend;
- banco;
- host de teste;
- criação do primeiro host;
- template Linux.

A documentação oficial inclusive descreve o fluxo de instalação do servidor/frontend, criação do host e associação do template Linux. ([Zabbix][1])

Definition of Done:

> Zabbix mostra pelo menos uma máquina Linux como `UP` e recebendo métricas.

---

2. EPIC 2 — Agente de armazenamento

Essa é a parte mais importante para não deixar o trabalho virar simplesmente "instalamos Zabbix".

Tarefa 4 — Criar gerador de carga

Vocês já tinham definido `fio` na Sprint 1.

Então, inicialmente, não precisam reinventar o benchmark.

Criem alguns perfis:

Perfil A — Sequencial

```
4 KB
read
sequential
```

Perfil B — Aleatório

```
4 KB
read
random
```

Perfil C — Escrita

```
128 KB
write
sequential
```

Perfil D — Concorrência

Variar:

```
numjobs
iodepth
```

Não precisa fazer todos na Sprint 2. Eu faria **2 ou 3 perfis simples** para provar que o pipeline funciona.

---

3. EPIC 3 — Métricas

Aqui vocês precisam separar:

Métricas que o sistema operacional/Zabbix consegue fornecer

- CPU;
- memória;
- utilização do dispositivo;
- I/O;
- temperatura, quando disponível;
- etc.

Métricas produzidas pelo teste

- IOPS;
- MB/s;
- latência;
- p99;
- p99.9;
- configuração da carga.

Essas métricas estão alinhadas ao que vocês já definiram na Sprint 1.

Tarefa 5 — Definir modelo das métricas

Antes de programar, definam uma tabela:

Métrica	Origem	Unidade
IOPS	fio/agente	operações/s
Throughput	fio/agente	MB/s
Latência média	fio/agente	ms
p99	fio/agente	ms
p99.9	fio/agente	ms
CPU	Zabbix	%
Temperatura	Zabbix/agente	°C
Utilização do disco	Zabbix	%

Isso vai ser MUITO útil para o artigo depois.

---

4. EPIC 4 — Enviar métricas para o Zabbix

Aqui eu faria o primeiro "momento de verdade" da Sprint.

Um fluxo conceitual:

```
fio
 ↓
resultado JSON
 ↓
agente
 ↓
extrai:
IOPS
BW
latência
p99
 ↓
```

```
zabbix_sender
 ↓
Zabbix Server
 ↓
gráfico
```

O `zabbix_sender` foi justamente feito para enviar dados de desempenho para o Zabbix Server, normalmente em conjunto com itens do tipo *trapper*. ([Zabbix][3])

Por exemplo, vocês poderiam ter chaves conceituais como:

```
storage.iops
storage.bandwidth
storage.latency.avg
storage.latency.p99
storage.latency.p999
storage.temperature
```

E cada execução do benchmark alimentaria esses valores.

---

5. EPIC 5 — Primeiro experimento

Aqui eu colocaria uma entrega bem concreta.

Experimento 1

Executar:

```
Carga A → coletar métricas
Carga B → coletar métricas
```

Por exemplo:

```
Teste 1
4 KB
leitura
sequencial

Teste 2
4 KB
leitura
aleatória
```

E produzir:

```
            Sequencial       Aleatório

IOPS            X                Y
MB/s            X                Y
Latência        X                Y
p99             X                Y
CPU              X                Y
```

O objetivo **não é ainda tirar grandes conclusões sobre SSDs**.

O objetivo é provar:

> "Quando mudamos o regime de carga, nosso sistema consegue observar uma mudança nas métricas."

Isso já constitui um resultado preliminar bastante defensável.

---

6. EPIC 6 — Evidências para o artigo

Essa parte eu faria **durante a sprint**, não no final.

Criem uma pasta:

```
docs/
├── architecture/
├── experiments/
├── screenshots/
├── results/
└── sprint-2/
```

E guardem:

```
zabbix-dashboard.png
zabbix-host.png
fio-test-01.json
fio-test-02.json
experiment-01.csv
experiment-02.csv
architecture.png
```

Assim vocês não chegam no dia da entrega tentando lembrar:

> "qual era mesmo aquele gráfico que a gente tinha?"

---

7. Divisão para 5 pessoas

Como vocês são 5, eu faria assim:

Pessoa	Responsabilidade principal
P1	Zabbix Server + infraestrutura
P2	Agente / código C/C++
P3	FIO + experimentos + configuração de cargas
P4	Métricas + integração Zabbix
P5	Artigo + metodologia + resultados

Mas com uma regra:

todo mundo precisa ter pelo menos alguma contribuição técnica no GitHub.

Porque senão fica:

> João fez código Maria fez Zabbix Pedro escreveu artigo Carlos e José fizeram slides

e o histórico de commits não demonstra participação equilibrada.

---

8. GitHub: eu faria as issues assim

Algo próximo de:

```
Sprint 2

[SETUP] Criar ambiente Docker
[SETUP] Configurar Zabbix Server
[SETUP] Configurar primeiro host Linux

[AGENT] Estruturar agente de monitoramento
[AGENT] Executar fio pelo agente
```

```
[AGENT] Parser dos resultados do fio

[METRICS] Definir modelo das métricas
[METRICS] Criar itens customizados no Zabbix
[METRICS] Implementar envio via zabbix_sender

[EXPERIMENT] Criar carga sequencial
[EXPERIMENT] Criar carga aleatória
[EXPERIMENT] Executar primeiro experimento
[EXPERIMENT] Exportar resultados

[DOC] Escrever seção de implementação
[DOC] Escrever resultados preliminares
[DOC] Registrar dificuldades
[DOC] Atualizar arquitetura
```

E cada pessoa abre PR/commit relacionado a uma dessas issues.
